

Student Academic Management System

Project Report for Flipped Course Evaluation

VITyarthi - Build Your Own Project

Course Name:	Programming in Java
Student Name:	Ayush Chouhan
Registration No:	25BAI10415
Department:	Computer Science & Engineering (AIML)
Submission Date:	September 18, 2026

Vellore Institute of Technology (VIT)

2. Introduction

The **Student Academic Management System** is an interactive, menu-driven command-line application implemented in Java to automate the processing and tracking of student academic profiles. Academic tracking typically introduces unnecessary overhead when dealing with disparate record ledgers, manual score compilations, and attendance monitoring. This project provides a robust, object-oriented software solution capable of recording student profiles, computing academic marks and grades, calculating attendance percentages, and persistently serializing academic reports to the local file system using Java I/O streams.

3. Problem Statement

Manual maintenance of student records via flat physical registers or unstructured spreadsheets often causes grade inaccuracies, attendance discrepancies, and loss of historical academic records. Existing lightweight tools frequently lack structured OOP design, runtime input fault tolerance, or persistent local storage. The **Student Academic Management System** solves these issues by delivering a structured console system that automates record management, enforces validation, executes background reporting, and ensures local flat-file persistence.

4. Functional Requirements

The project satisfies the core requirements through four functional modules:

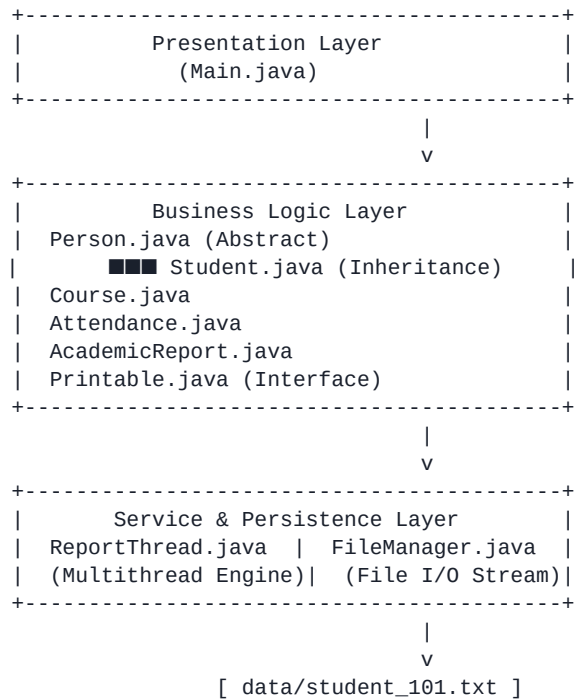
- **Student Management Module:** Allows adding new student records (Student ID, Name, Department), viewing all active profiles, and searching, updating, or deleting records by ID.
- **Academic Management Module:** Records subject course names and marks (validated within 0–100), computes aggregate and average marks, and assigns letter grades (e.g., Grade A).
- **Attendance Management Module:** Tracks total classes held and attended, computes overall percentage, and displays status (e.g., "Satisfactory" for $\geq 75\%$).
- **Reporting & File Management Module:** Assembles academic summaries using multithreading (ReportThread), saves records to `data/student_<id>.txt` via FileManager, and reads saved records from disk.

5. Non-Functional Requirements

- **Error Handling & Fault Tolerance:** Employs defensive try-catch structures to catch invalid non-numeric inputs (e.g., entering "abc" into numeric prompts) and prevent runtime application crashes.
- **Performance & Concurrency:** Asynchronous report compilation is offloaded to a separate background Java thread (ReportThread) to decouple computational tasks from menu execution.
- **Reliability & Data Persistence:** File streams write records to the `data/` folder, ensuring persistent record retention across program restarts without requiring external database setups.
- **Maintainability & Modularity:** Decoupled across 8 distinct Java source classes/interfaces within `src/`, adhering strictly to single-responsibility OOP principles.

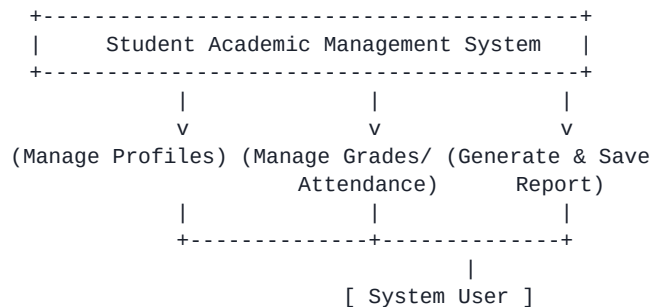
6. System Architecture

The system follows a three-tier architecture separating interface presentation, core domain logic, and file persistence:



7. Design Diagrams

7.1 Use Case Diagram



7.2 Workflow Diagram

```
[Start]
--> [Display CLI Menu Options 1-13]
--> [Prompt User Input]
--> [Validate Input (try-catch)]
-->
    |-- (1-5) --> Execute Student Management Operations
    |-- (6-7) --> Add Course Marks / Update Attendance Records
    |-- (8)    --> Spawn ReportThread & Generate Performance Report
    |-- (9-10) --> Persist to data/student_<id>.txt / Read from disk
    |-- (11)   --> Demonstrate Runtime Polymorphism
    |-- (12)   --> Calculate Factorial Loop Routine
    |-- (13)   --> Exit Application loop
--> [Return to Menu / Terminate]
```

7.3 Sequence Diagram

User	Main.java	ReportThread	FileManager	Filesystem
	-- Select (8,9) -->			
	-- start() ----->			
		-- run()		
		-- Compile Report		
	<- Report Ready -			
	-- saveToFile(data) ----->			
			-- write() ---->	
			<- Ack -----	
<- Output Confirm-	-- Write Success -----			

7.4 Class Diagram

```
+-----+
| <<abstract>> |
| Person |
+-----+
| - id: int |
| - name: String |
| - department: String |
| + displayInfo()* |
+-----+
      ^
      | extends
      |
+-----+
| Student |
+-----+
| - courses: List<Course>
| - attendance: Attendance
| + addCourse(c: Course): void
| + setAttendance(a: Attendance): void
| + calculateAverage(): double
| + displayInfo(): void
+-----+

<<interface>>
+-----+
| Printable |
+-----+
| + displayInfo() |
+-----+
      ^
      | implements
      |
+-----+
| Student |
+-----+
```

8. Design Decisions & Rationale

- **Abstract Class (Person):** Standardizes basic identity fields (id, name, department) for reusability while laying the foundation for future extensions (e.g., instructors, teaching assistants).
- **Interface (Printable):** Establishes a contract for formatted console output across domain models without creating direct inter-class dependencies.
- **Flat-File Persistence (FileManager.java):** Provides zero-configuration local persistence using plain text files inside data/, keeping the setup lightweight and self-contained.
- **Multithreading Model (ReportThread.java):** Demonstrates concurrent task execution in Java by isolating computational report assembly from the main thread loop.

9. Implementation Details

Java Concept	Implementation in Project
Classes & Objects	Domain models structured into Student, Course, Attendance, and AcademicReport.
Inheritance & super	Student extends abstract Person, utilizing super(studentId, name) constructor delegation.
Polymorphism	Dynamic runtime dispatch: Person person = students.get(0); person.displayInfo();
Interface Contract	Printable interface implemented by Student to enforce display formatting.
Multithreading	ReportThread extends Thread executes asynchronous background report rendering.
File I/O Streams	FileManager handles disk persistence to data/student_<id>.txt and loads historical files.
Exception Handling	try-catch guards against invalid numeric input and missing file references.

10. Screenshots / Results

```
----- MENU -----
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Add Course Marks
7. Add Attendance
8. Generate Academic Report
9. Save Student Record
10. Read Saved Student Record
11. Demonstrate Runtime Polymorphism
12. Calculate Factorial
13. Exit
-----
Enter your choice: 
```

Figure 1: Main Menu of the Student Academic Management System

```
Enter Student ID: 101

=====
                        ACADEMIC PERFORMANCE REPORT
=====
Student ID   : 101
Student Name : ayush
Department   : cse

Course Performance:
  java : 86.0
  Calculus : 89.0

Total Marks   : 175.0
Average Marks : 87.5
Overall Grade : A

Attendance:
  Total Classes   : 40
  Classes Attended : 32
  Percentage      : 80.0%
  Status          : Satisfactory
=====
```

Figure 2: Academic Performance Report for Student ID 101

```
Runtime Polymorphism Demonstration
Person reference is referring to a Student object.
Student ID   : 101
Student Name : ayush
Department   : cse
```

Figure 3: Runtime Polymorphism Demonstration (Person reference to Student object)

```
Enter Student ID: 101
Student record saved successfully.
```

Figure 4: Successful save of student record to disk

```
Enter your choice: abc
Please enter a valid number.
```

Figure 5: Input validation – non-numeric input handled gracefully

11. Testing Approach

The application underwent both functional testing and input validation boundary tests:

- **Functional Test:** Added student ID 101 ("ayush", cse), logged marks for Java (86.0) and Calculus (89.0), and recorded 32/40 attendance sessions. The system accurately computed a total of 175.0, average 87.5, assigned Grade A, marked attendance as 80.0% (Satisfactory), and saved the record to disk successfully.
- **Input Validation Test:** Passed non-numeric string "abc" to numeric menu selections. Caught by defensive try-catch handlers without crashing the program (see Figure 5).
- **Boundary Test:** Passed negative integer "-8" to the factorial calculator; the system caught the out-of-bounds input and reprompted the user.
- **Polymorphism Test:** Option 11 demonstrated runtime polymorphism with a Person reference pointing to a Student object (see Figure 3).

12. Challenges Faced

- **Scanner Buffer Management:** Encountered newline consumption bugs when alternating between `nextInt()` and `nextLine()`, resolved by adding explicit buffer clearing.
- **Thread Coordination:** Ensured that `ReportThread` completed processing before `FileManager` attempted to access the compiled report string.
- **Cross-Platform File Paths:** Normalized file directory paths to ensure consistent file creation across Windows and Unix environments.

13. Learnings & Key Takeaways

- Gained practical experience designing an 8-class modular system using core OOP concepts (inheritance, polymorphism, interfaces).
- Acquired hands-on proficiency in robust console exception handling and defensive programming.
- Learned how to implement basic multithreading in Java and handle persistent storage using file streams.

14. Future Enhancements

- **Database Migration:** Replace text files with an embedded SQLite or PostgreSQL database using JDBC for relational queries.
- **Graphical Interface:** Implement a GUI using JavaFX or Swing to replace the terminal menu.
- **Role-Based Access Control:** Add user login authentication with separate permissions for faculty and students.

15. References

- Oracle Java SE Documentation: The Java™ Tutorials
- Horstmann, C. S. (2022). *Core Java Volume I – Fundamentals* (12th Edition). Prentice Hall.
- Schildt, H. (2018). *Java: The Complete Reference* (11th Edition). McGraw-Hill Education.